RoboCerebra Future Plan: Subtask-Level Failure Detection and Recovery

Prepared as a short technical note on how the RoboCerebra benchmark can be used to identify the failed subtask in a long-horizon rollout, then restart from the previous GT subtask boundary or ask the robot to return to the GT pose.

> **Executive summary.** RoboCerebra is useful not only because it reports whole-episode success, but because its evaluation structure can expose where a long-horizon execution first fails. Each task is decomposed into ordered step descriptions, each step can be linked to a demonstration boundary state, and the goal predicates in `goal.json` can be assigned to task steps. Therefore, if the predicate expected at subtask k does not become true within the step-k horizon, or if a predicate completed before k regresses, the benchmark can label subtask k as the failure point.

1. Benchmark Structure Used for Diagnosis

The evaluator already contains the pieces needed for subtask-level diagnosis. The file `task_description.txt` defines the ordered subtask instructions and their demonstration frame ranges. The evaluator loads those start indices from the demonstration states in `demo.hdf5`. In parallel, `goal.json` stores object-state relations, and the newer format includes a `task_step` annotation. This makes it possible to map each success predicate to the subtask that is supposed to achieve it.

The important point is that a rollout does not need to be judged only at the final timestep. At each step boundary, the evaluator can call `env._check_success(goal)` and compare the number and identity of completed predicates against the expected predicates for the current subtask. This turns a single whole-task failure into a structured diagnosis: completed prefix, first failing subtask, missing predicate, and any regression of earlier progress.

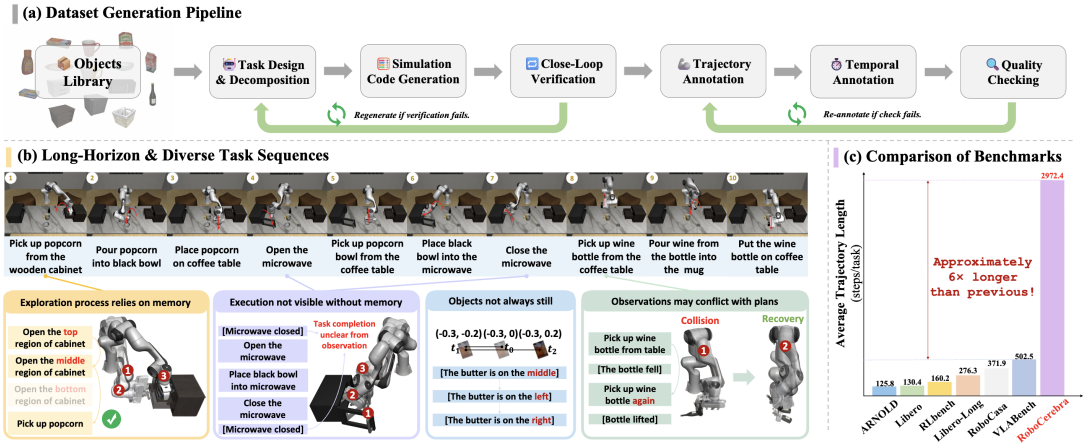

Figure 1. RoboCerebra decomposes long-horizon robot tasks into step sequences; this structure is the basis for subtask-level diagnosis.

2. Detecting the Failed Subtask

For subtask k, I will define success as the transition from the predicate set before k to the predicate set expected after k, while maintaining all predicates that were completed before k. A failure is assigned to k when the expected predicate does not change, changes in the wrong direction, or takes longer than the step horizon. The same measurement can be used in the Ideal, Random_Disturbance, Observation_Mismatching, and Mix settings.

Signal	Computation	Failure interpretation
Predicate progress	Compare `env._check_success(goal)` before and after subtask k.	The expected predicate for subtask k is still false.
Regression	Check whether predicates completed before k become false after policy actions.	The policy damaged previous progress.
Time budget	Use the fixed segment horizon controlled by `switch_steps`.	The policy timed out at subtask k.
State deviation	Compare the current object and robot pose against the GT boundary state.	The rollout has moved off the demonstrated state manifold.
Perturbation response	Evaluate the same logic under Random_Disturbance, Observation_Mismatching, and Mix.	The model is not robust to controlled disturbance.

The monitor can be implemented with the following step-boundary loop:

```
for subtask k:
  progress_before = check_success(goal)
  run policy for the step-k horizon
  progress_after = check_success(goal)
  if expected_predicates[k] are not completed: label failure at k
  if previous_predicates regressed: label recovery-required regression
  log missing predicates, pose deviation, video clip, and recovery mode
```

This produces a more useful dataset than final success alone. A failed episode can still contribute the completed prefix, the exact first failure location, and a controlled starting state for recovery experiments.

3. Recovery Protocols After Failure

After identifying the failing subtask, I propose evaluating two recovery protocols. They should be reported separately because they answer different questions: one is a clean diagnostic reset, and the other is a physically meaningful recovery attempt.

Recovery mode	Procedure	What it tests
A. GT-state restart	Set the simulator to the saved demonstration state at the boundary after subtask k-1. Reset the policy queue/state, mark prior predicates complete with the resume handler, and restart from the step-k instruction.	Clean per-subtask competence when previous progress is guaranteed correct.
B. Robot-to-GT-pose recovery	Command the robot back to the GT EEF or joint pose at the previous boundary. Recheck prior predicates and continue from subtask k; use full GT-state restart only if the world is invalid.	Embodied recoverability under a more realistic robot protocol.

The GT-state restart should be treated as the controlled upper-bound condition. In the current evaluator, it corresponds to setting the simulator to the saved step state and using the resume handler to mark all prior subtasks as logically complete. This isolates whether the model can solve subtask k when the previous subtasks are correct.

The robot-to-GT-pose condition is closer to the real robot setting. Instead of teleporting the entire simulator state, the robot is asked to move back to the GT end-effector or joint pose at the previous boundary. Prior predicates are then checked again. If the world is still valid, the policy resumes from the failing subtask; if the world is invalid, the evaluator can fall back to GT-state restart and record that the physical recovery failed.

4. Experiment Plan

The next implementation step is to add a per-subtask attempt record to the evaluator output.

- For each subtask, store the start state index, instruction text, expected predicates, completed predicates, missing predicates, segment video path, and failure type.
- Run three conditions: normal chained rollout, isolated subtask rollout from the GT previous boundary, and recovery rollout after a detected failure.
- Compare task regimes: Ideal, Random_Disturbance, Observation_Mismatching, Mix, Memory_Execution, and Memory_Exploration.
- Report first-failing-subtask distribution, per-subtask isolated success, GT-state-restart recovery, robot-to-GT-pose recovery, and the chained-vs-isolated gap.

The resulting analysis should show whether the model's main limitation is high-level planning, low-level execution, memory over a long horizon, or inability to recover after the world state drifts away from the demonstration distribution.

5. Current Evidence

The local checkpoint summaries already track subtask completion independently from episode success. This is evidence that the current infrastructure can measure partial progress and can be extended into a more explicit subtask-failure diagnostic benchmark.

Model	Step-0 subtask	Best checkpoint	Latest checkpoint in CSV
PI0	16.6%	19.9% at 25,000 steps	18.1% at 125,000 steps
PI0.5	14.4%	19.6% at 35,000 steps	17.9% at 100,000 steps

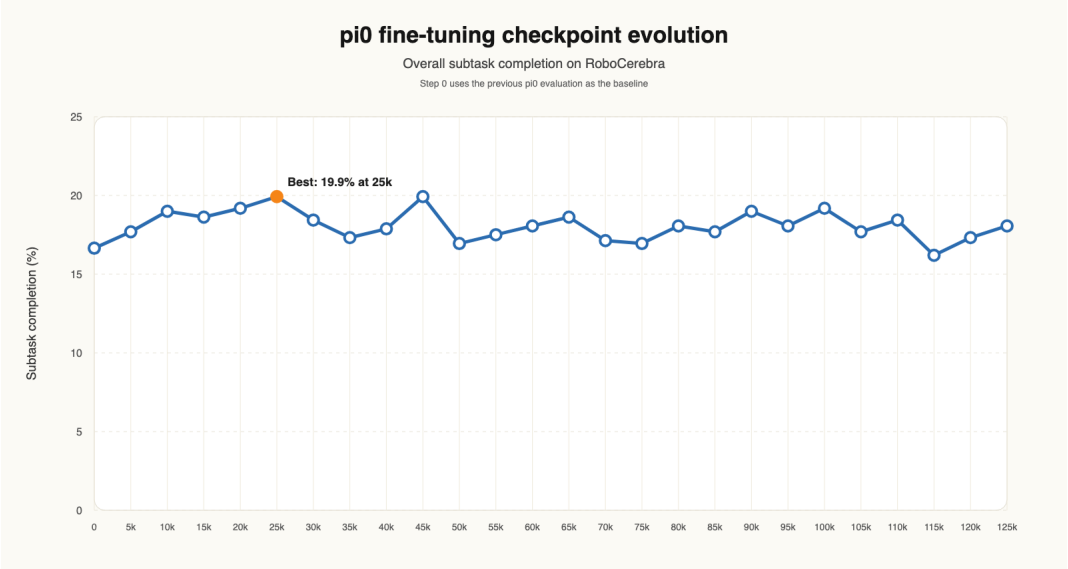

Figure 2. Example local checkpoint plot: overall subtask completion is tracked separately from whole-episode success.

6. Implementation Hooks in the Current Evaluator

The proposed changes do not require replacing the evaluator. They can be added around the existing step parsing, resume, segment transition, and logging logic.

Current component	How it supports the plan
`load_actions_with_steps`	Loads each goal predicate and its `task_step`, which is the key mapping from predicate success to subtask index.
`create_step_based_resume_handler`	Builds the set of subtasks that should already be complete before a resumed step.
`update_completion_tracking`	Already measures incremental subtask progress; this is the natural place to add missing-predicate and regression labels.
`handle_segment_transition`	Already switches between subtasks and can trigger GT-state restart after a detected failure.
`save_results_log`	Can be extended with a per-subtask attempt table while keeping the current aggregate success and subtask-rate metrics.

7. Immediate Deliverables

- Patch the evaluator logs to emit one record per subtask attempt.
- Add flags for GT-state restart and robot-to-GT-pose recovery.
- Create three plots for the next report: first-failure histogram, recovery comparison, and chained-vs-isolated performance gap.
- Use these plots to decide whether future training should target planning, execution, or recovery behavior.

Takeaway. RoboCerebra can be used as a diagnostic benchmark, not only a final-score benchmark. The benchmark can identify the failed subtask by comparing predicate progress at step boundaries, then test two recovery conditions: restart from the GT boundary after the previous subtask, or ask the robot to move back to the GT pose and continue.